

Enhanced Hyperkinetic Learning Binary Dragonfly Algorithm for Optimal Feature Selection

ABSTRACT

The "curse of dimensionality," produced by the fast proliferation of information science, will have a detrimental impact on the machine learning model's performance. In the data mining process, feature selection is often thought of as a pre-processing technique for finding an optimal subset of features from a given collection of all features. An enhanced Hyperkinetic Learning Binary Dragonfly Algorithm (HLBDA) is provided in this article as a wrapper-based strategy for finding the best subset of features for a particular classification task. Enhanced HLBDA is an improved version of the Binary Dragonfly Algorithm (BDA) in which the voting classifier, with estimators such as logistic regression, decision tree, and SVM, is combined with modifications to the Levy Flight Mechanism (LFM), food, and predator to improve the algorithm's randomisation, probabilistic behaviour, artificial dragonfly, discovery, and searching behaviour. The suggested enhanced HLBDA is compared to nine existing algorithms. Over eighteen datasets from the University of California, Irvine (UCI) collection, many assessment indicators are used to evaluate and compare the performance of various techniques. The findings show that enhanced HLBDA is superior in terms of enhancing prediction accuracy and minimising the number of selected features.

I. INTRODUCTION

Data mining is one of the fastest developing sectors of information technology in recent years, owing to the massive quantity of data produced each minute as well as the necessity to translate that data into meaningful knowledge [1–4]. Data mining and knowledge extraction approaches have grown in popularity as standard machine learning [5] and deep learning [6–8] have become more popular over time, as most of these learning techniques are hungry for useful data. Data pre - processing, knowledge discovery, and pattern evaluation are all part of data mining [9]. Feature selection is one of the most important data pre-processing stages (FS). FS is described as the process of determining the best subset of features to reflect the original feature set while maintaining a high level of classification accuracy [9]. We may eliminate duplicate and unnecessary features from datasets using FS. Irrelevant and duplicated characteristics in the learning process not only deceive the learning algorithm and degrade performance, but they also add to computational complexity and storage requirements [10]. Filter and wrapper techniques are the two basic kinds of FS techniques [11–15]. Statistical approaches are used to analyse a feature subset or detect correlation between variables in filter methods, which do not require any learning algorithms. Wrapper models, on the other hand, evaluate feature subsets using learning algorithms similar to classifiers. Although filters are quicker than wrappers, wrappers can achieve greater classification accuracy than filters [16].

To address the constraints of standard optimisation approaches, numerous meta-heuristic optimisation strategies have been presented in the field of FS over the last two decades [17]. In general, nature-inspired algorithms have demonstrated good performance in addressing search problems [17]. Many approaches for simulating animal, bird, fish, and wolf behaviour have been presented in the literature. Genetic Algorithm (GA) [18] is the first suggested FS algorithm that employs a meta-heuristic approach. The authors suggest a chaotic genetic features extraction optimisation approach (CGFSO) in [19]. Particle Swarm Optimization (PSO) is based on the social synergy of a flock of birds and first proposed in [20]. [21–23] proposes a number of FS techniques based on PSO. Another meta-heuristic method, Ant Colony Optimization (ACO) [24], simulates the behaviour of actual ants when looking for the shortest path to a food source. [25, 26] proposes ACO-based FS methods. Artificial Bee Colony (ABC) optimisation method [27] and its FS counterpart [28] are also presented, based on the biological behaviours of bees. The Social Mimic Optimization (SMO) Algorithm [14] is presented by emulating someone who is more educated, esteemed, or powerful, and its FS equivalent is proposed in [29]. Grey Wolf Optimizer (GWO) [16], Whale Optimization Algorithm (WOA) [30], Ant Lion Optimizer (ALO) [17], Gravitational Search Algorithm (GSA) [31], Social Ski-driver (SSD) Algorithm [32], Sailfish Optimizer (SFO) [33], Ring Theory-based Evolutionary Algorithm (RTEA) [34] are some recent optimisation algorithms and their FS counterparts.

Because of the relevance of metaheuristic algorithms and the fact that they are employed in so many applications, researchers have begun to write survey articles on them, such as a comprehensive and meta-analysis study of the whale optimisation algorithm [48]. [47] offered an overview of the latest generation of metaheuristic algorithms. [45] offered another survey on nature-inspired metaheuristic algorithms with domain criteria. [46] offered a recent development on changes to the cuckoo search (CS) method. Yang and Deb [44] were the first to build the CS. It uses the Levy flying movement of some fruit flies and birds to simulate cuckoo species' brood parasite behaviour.

The major goal of this research is to offer a wrapper feature selection technique that employs the current BDA algorithm as a search strategy with modification in levy flight, food, predator and the voting classifier in which the estimators are logistic regression, decision tree and SVM as evaluators. The remainder of the paper is laid out as follows: The BDA algorithm is described in section 2. The details of the suggested technique are explained in section 3. Section 4 contains the experimental data, whereas Section 5 and 6 contains the conclusions and future plans.

There are several major contributions in this study, including:

- Improved HLBDA: This is an upgraded version of BDA's regular binary version.
- This work tackle the local optima problem of regular BDA is the most significant advance.
- Improved HLBDA was analysed and compared to several well-known algorithms, including GA [53], PSO [53], DF [53], BGWOPSO [54], WOSAT-2[54], BPSO [55], BALO [55], BGWO [55], and BDA_SA [55].

– We compared the results to those obtained with all features, using 18 data sets from the UCI library that are often used in feature selection research utilising 18 benchmark data sets from the UCI library. The advantage of improved HLBDA over these baseline algorithms is clearly demonstrated by these results.

II. RELATED WORK

Data mining and data analysis have shown the potential utility of large data in the training process as part of popular information technologies [56]. Particle swarm optimisation (PSO) and its binary variation (BPSO) are two wrapper-based algorithms that have been widely employed in feature selection research. Two recent techniques in wrapper feature selection were the self-adaptive parameter and strategy based PSO [57] and enhanced BPSO [58]. Recently, writers in [59] proposed a multi-swarm technique for dividing all particles into multiple tiny size sub-swarms, which were then advanced using a local form of PSO. Furthermore, the suggested PSO technique used an elite learning mechanism that enabled information to flow across sub-swarms, effectively increasing population diversity.

Grey wolf optimizer (GWO) is a new metaheuristic algorithm that simulates the natural leadership structure and hunting behaviour of grey wolves [60]. Recently, [61, 62] have employed the GWO and its binary equivalents (BGWO) as the wrapper approach for feature selection. The suggested GWO variants outperformed other optimizers in their experiments. Furthermore, Chantar et al. [63] included an elite crossover mechanism for Arabic text categorisation into BGWO. Furthermore, Too and Abdullah [64] integrated opposition-based learning into a competitive grey wolf optimizer to increase hand motion categorisation prediction accuracy. In addition, writers in [65] proposed combining the grey wolf optimizer with the biphasic mutation operator to evolve the leader's character during the search process.

For feature selection, another idea was built based on the whale optimisation algorithm (WOA). Mafarja and Mirjalili [66], for example, suggested a WOA-based feature selection strategy in which the tournament selection mechanism was used to replace the random mechanism in the selection process. Mafarja and Mirjalili combined the WOA with a simulated annealing approach in [67], which improved the program's capacity to explore the feature space. Tubishat et al. [68] have presented an enhanced WOA to address the issue of feature selection in Arabic sentiment analysis. They used information gain to filter out certain undesired aspects before introducing an unique strategy based on opposition-based learning principles to develop new opposing solutions.

Mirjalili recently created the dragonfly algorithm (DA) for global optimisation purposes. He then created a V-shaped transfer function, which he implemented into DA to allow dragonflies to search in the binary search space [69]. The binary dragonfly algorithm (BDA) was used by the authors in [70] to pick features. However, the BDA, like other metaheuristic algorithms, has the drawback of confining local optima. To improve the convergence rate, Sayed et al. [71] included the chaos theorem into BDA. They found that the Gauss map outperformed the other eight chaotic maps in terms of improving learning model performance. Furthermore, an improved BDA technique for newborn cry categorisation was developed. The suggested model used an elitism mechanism to save the prospective dragonfly and then insert it to replace the population's worst dragonfly, which improved the algorithm's convergence behaviour in discovering interesting solutions.

III. DRAGONFLY ALGORITHM

III.I Exploration and exploitation operators

The DA algorithm was inspired by dragonflies [35], as the name suggests. This method may be thought of as a swarm intelligence technique for estimating the global optimum of a problem. The following are the mathematical models for implementing dragonfly swarming behaviour: [36]:

- Separation refers to the strategy that individuals use to avoid colliding with their neighbours. The mathematical model for this behaviour is Eq (1).

$$S_i = - \sum_{j=1}^N X - X_j \quad (1)$$

where X is the current individual's location, X_j is the j -th adjacent individual of the current individual's position, and N is the community size.

- Alignment refers to an individual's velocity matching that of nearby persons. The mathematical model for this behaviour is Eq (2).

$$A_i = \frac{\sum_{j=1}^N V_j}{N} \quad (2)$$

where V_j is the velocity of the j -th neighbour and N is the size of the neighbourhood

- Individuals' propensity to the neighborhood's centre of mass is referred to as cohesion. The mathematical model for this behaviour is Eq (3).

$$C_i = \frac{\sum_{j=1}^N X_j}{N} - X \quad (3)$$

where X is the current individual's location, X_j is the j -th adjacent individual of the current individual's position, and N is the neighbourhood size.

- Attraction towards the food source and escaping from enemies are other two key behaviors that each individual behaves to survive. The attraction towards the food is modelled as in Eq. (4).

$$F_i = X^+ - X \quad (4)$$

where X^+ represents the position of the food source and X represents the current individual's position.

$$E_i = X^- + X \quad (5)$$

where X^- represents the enemy's position and X represents the current individual's position.

In this research, dragonfly behaviour is supposed to be a mixture of these five corrective patterns. Two vectors are used to update the position of artificial dragonflies in a search space and replicate their movements: step (ΔX) and position (X). In PSO, the step vector corresponds to the velocity vector, and the DA method is built on the framework of the PSO algorithm. The step vector represents the direction of the dragonflies' movement and is described as follows (notice that the artificial dragonfly position update model is specified in one dimension, but the introduced approach may be expanded to higher dimensions):

$$\Delta X_{t+1} = (sS_i + aA_i + cC_i + fF_i + eE_i) + w\Delta X_t \quad (6)$$

where s is the separation weight, and S_i represents the i -th individual's separation, a represents the alignment weight, A_i represents the i -th individual's alignment, and c represents the cohesion weight. C_i is the i -th individual's cohesiveness, f is the food factor, F_i is the i -th individual's food source, e is the enemy factor, E_i is the i -th individual's enemy location, w is the inertia weight, and t is the iteration number.

The position vectors are calculated once the step vector has been determined:

$$X_{t+1} = X_t + \Delta X_{t+1} \quad (7)$$

where t is the current iteration.

Different explorative and exploitative behaviours may be achieved during optimisation using separation, alignment, cohesion, food, and adversary components (s, a, c, f, and e). Because dragonfly neighbours are so crucial, each artificial dragonfly is considered to have a neighbourhood with a particular radius.

Only two forms of swarms are seen in dragonflies: static and dynamic. However, in a static swarm, alignments are very low but cohesiveness is quite strong in order to assault preys. As a result, when exploring the search space, we allocate dragonflies with high alignment and low cohesion weights, and when exploiting the search space, we assign dragonflies with low alignment and high cohesion weights. The radius of neighbourhoods is raised according to the number of iterations while transitioning from exploration to exploitation. Adaptively tuning the swarming factors (s, a, c, f, e, and w) during optimisation is another technique to balance exploration and exploitation.

The artificial dragonflies are required to fly about the search space using a random walk (Levy flying) when there are no neighbouring solutions to increase their unpredictability, stochastic behaviour, and exploration. The location of dragonflies in this scenario is updated using the equation:

$$X_{t+1} = X_t + Levy(d) \times X_t \quad (8)$$

where t is the current iteration, and d is the dimension of the position vectors.

The Levy flight is calculated as follows [37]:

$$Levy(x) = 0.01 \times \frac{r1 \times \sigma}{|r2|^{\frac{1}{b}}} \quad (9)$$

where r1, r2 are two random numbers in [0,1], b is a constant (equal to 1.5 in this work), and r is calculated as follows:

$$\sigma = \left(\frac{\Gamma(1+\beta) \times \sin(\frac{\pi\beta}{2})}{\Gamma(\frac{1+\beta}{2}) \times \beta \times 2^{\frac{\beta-1}{2}}} \right)^{\frac{1}{\beta}} \quad (10)$$

Where $\Gamma(x) = (x - 1)!$

III.II The DA algorithm for single-objective problems

The DA method begins the optimisation process by generating a set of randomly generated solutions for a particular optimisation issue. In fact, random values defined inside the lower and higher ranges of the variables are used to initialise the position and step vectors of dragonflies. The location and step of each dragonfly are updated in each cycle using Eqs. (7)/(8) and (6). The neighbourhood of each dragonfly is picked for updating the X and ΔX vectors by computing the Euclidean distance between all the dragonflies and picking N of them. The procedure of updating positions is repeated recursively until the end criterion is met. Algorithm 1 contains the pseudo-codes for the DA algorithm.

Algorithm 1. Pseudo-code of the DA algorithm
<i>Initialize the dragonflies population X_i ($i = 1, 2, \dots, n$)</i> <i>Initialize step vectors ΔX_i ($i = 1, 2, \dots, n$)</i> while the end condition is not satisfied <i>Calculate the objective values of all dragonflies</i> <i>Update the food source and enemy</i> <i>Update w, s, a, c, f, and e</i> <i>Calculate S, A, C, F, and E using Eqs. (1) to (5)</i> <i>Update neighbouring radius</i> if a dragonfly has at least one neighbouring dragonfly <i>Update velocity vector using Eq. (6)</i> <i>Update position vector using Eq. (7)</i> else <i>Update position vector using Eq. (8)</i> end if <i>Check and correct the new positions based on the boundaries of variables</i> end while

The consideration of separation, alignment, cohesion, attraction, distraction, and random walk in this study is one of the fundamental distinctions between the DA and PSO algorithms. Although there have been some attempts in the literature to include separation,

alignment, and cohesiveness into PSO [38–40], this article mimics dragonfly swarming behaviour by taking into account all conceivable characteristics that may be applied to individuals in a swarm. Swarms, both static and dynamic, are also innovative notions. This work's suggested model is also somewhat distinct from the present enhanced PSO.

III.III The DA algorithm for binary problems (BDA)

A binary search space is quite different from a continuous search space in terms of optimisation. The search agents of DA can update their locations in continuous search spaces by adding step vectors to the position vectors. In a binary search space, however, adding step vectors to X will not update the location of search agents since position vectors can only be given by 0 or 1. The existing approaches for addressing binary issues in the literature are easily adaptable to this algorithm due to the similarities of DA and other SI techniques.

A transfer function, according to Mirjalili and Lewis [41], is the simplest and most successful way to transform a continuous SI methodology to a binary algorithm without changing the structure. Transfer functions take velocity (step) data as inputs and output a number between 0 and 1, indicating the likelihood of changing locations. The value of the velocity vector is exactly proportional to the output of such functions. As a result, a significant velocity value for a search agent indicates that it is very likely to update its position. This approach, like continuous optimisation, replicates rapid changes in particles with enormous velocity values (Fig. 6). Figure 6 shows two instances of transfer functions from the literature [41–43].

The v-shaped transfer functions are superior than the s-shaped transfer functions, according to Saremi et al. [43], since they do not compel particles to accept values of 0 or 1. The following transfer function is used to solve binary issues using the BDA method [41]:

EQUATION 11

Binary Dragonfly Algorithm for Feature Selection (binary dragonfly (1)) paper. 6 pages one

$X_{t+1} = \begin{cases} \neg X_t, & x < T(\Delta x_{t+1}) \\ X_t, & x \geq T(\Delta x_{t+1}) \end{cases}$

$$X_{t+1} = \begin{cases} -X_t & r < T(\Delta x_{t+1}) \\ X_t & r \geq T(\Delta x_{t+1}) \end{cases}$$

where r is a random integer between 0 and 1, and $T(\Delta x_{t+1})$ is calculated as in Eq. (12).

$$T(\Delta x) = \left\lfloor \frac{\Delta x}{\sqrt{\Delta x^2 + 1}} \right\rfloor \quad (12)$$

For all artificial dragonflies, this transfer function is first used to calculate the chance of shifting location. To update the location of search agents in binary search spaces, the following new updating position formula is used:

The BDA method will be able to handle binary issues quickly with the transfer function and new position update equations, provided the problem is properly formulated. Because the distance between dragonflies in a binary space cannot be determined as clearly as it can in a continuous space, the BDA algorithm treats all of the dragonflies as a single swarm and simulates exploration/exploitation by adaptively tuning the swarming factors (s , a , c , f , and e) as well as the inertia weigh (w). Algorithm 2 shows the pseudo-codes for the BDA algorithm.

Algorithm 2. Pseudo-code of the BDA algorithm

```

Initialize the dragonflies population  $X_i$  ( $i = 1, 2, \dots, n$ )
Initialize step vectors  $\Delta X_i$  ( $i = 1, 2, \dots, n$ )
while Maximum number of iterations is not met
    Evaluate the fitness values of dragonflies
    Update the food source,  $X_f$  and enemy,  $X_e$ 
    Update  $s$ ,  $a$ ,  $c$ ,  $f$ ,  $e$ , and  $w$ 
    for  $i = 1$  to number of dragonflies,  $N$ 
        Calculate  $S$ ,  $A$ , and  $C$  using (1), (2), and (3)
        Compute  $F$  and  $E$  using (4) and (5)
        Update step vector using (6)
        Update the position of dragonfly ( $i$ -th) using (11)
    end for
end while

```

IV. THE PROPOSED ALGORITHM

FS is a binary optimisation problem in which the solutions are limited to binary 0 and 1 values. As a result, the binary form of the DA method is appropriate for solving this issue. The solution of the issue is represented in this work by a vector of zeros and ones, where the zero element indicates that the related feature is not selected and the one element indicates

that this feature is selected. The number of features in the original dataset determines the length of the solution vector.

In this work, a wrapper feature selection method that based on the BDA algorithm is proposed. The voting classifier in which the estimators are logistic regression, decision tree and SVM along with modification in Levy Flight Mechanism (LFM) i.e.. equation 15, food i.e.. equation 13, predator i.e.. equation 14 are combinely utilised to improve the Binary Dragonfy Optimizer's best solution (subset of features) so found so far.

A unique binary dragonfly method is presented and applied to feature selection in this part. The unique binary dragonfly technique employs a Hyperkinetic learning strategy and improves step size in LFM.

In the position update process, Hyperkinetic Learning technique incorporates the notion of personal best and personal worst solutions. The dragonflies in the traditional BDA collaborate to attract and distract the global best solution (food source) and global worst solution (enemy). However, it is thought that by include the personal best and worst dragonflies in these activities, the possibility for food locating and adversary fleeing behaviours can be increased. Unlike BDA, the innovative binary dragonfly algorithm's attraction and distraction are determined using the following equations 13, 14.

$$F_i = \frac{(X_{pb_i} - X_i) + (X_f - X_i)}{2} \quad (13)$$

$$E_i = \frac{(X_{pw_i} + X_i) + (X_e + X_i)}{2} \quad (14)$$

where X_{pb} represents the personal best dragonfly's position, X_{pw} represents the personal worst dragonfly's position, X represents the dragonfly's position, X_f represents the food supply, and X_e represents the opponent.

During the search phase, the dragonfly may also learn from both personal best and global best solutions according to the Hyperkinetic learning technique.

LFM is named after Paul Levy, a French mathematician. This system is technically infinitely variable (possible length).

When there is no neighbourhood solution, a random walk (LFM) solution is used to increase the randomisation, probabilistic behaviour, and artificial dragonfly discovery. As a result, the positions of artificial dragonflies are updated as illustrated in equations (8),(9), and (10) respectively .

A multiplication was employed in the LFM used in DA, which was not included in the original mathematical formula of the flying technique. As seen in equation (9), this multiplication was produced by taking 1% of the LFM size.

The goal was to keep the step size under control. After LFM, this multiplication determines the value of a solution, which is the amount of the best individual deviation (position of the best individual).

Although LFM improves DA performance to some amount, it has the drawback of causing very lengthy steps depending on the mechanism's properties. These important phases in the algorithm are attempted to be controlled by using the step size as shown in equation (15). This way of solution is superior than the previous one.

$$Levy(x) = \left(i/N^2\right)^{(i+N)^{1/2}} \times \frac{r1 \times \sigma}{|r2|^{\frac{1}{\beta}}} \quad (15)$$

Where i is the current iteration, N is the total number of iterations, $r1$, $r2$ are two random numbers in $[0,1]$, β is a constant (equal to 1.5 in this work), and σ is calculated as follows:

$$\sigma = \left(\frac{\Gamma(1+\beta) \times \sin(\frac{\pi\beta}{2})}{\Gamma(\frac{1+\beta}{2}) \times \beta \times 2^{(\frac{\beta-1}{2})}} \right)^{\frac{1}{\beta}}$$

Where $\Gamma(x) = (x - 1)!$

Algorithm 3. Pseudo-code of the unique DA algorithm

```

Initialize the dragonflies population  $X_i$  ( $i = 1, 2, \dots, n$ )
Initialize step vectors  $\Delta X_i$  ( $i = 1, 2, \dots, n$ )
while Maximum number of iterations is not met
    for  $i = 1$  to number of dragonflies,  $N$ 
        Calculate the fitness value dragonfly ( $i$ -th)
        Update the personal best dragonfly,  $X_{pbi}$ 
        Update the personal worst dragonfly,  $X_{pwi}$ 
    end for
    Update the food source,  $X_f$  and enemy,  $X_e$ 
    Update  $s$ ,  $a$ ,  $c$ ,  $f$ ,  $e$ , and  $w$ 
    for  $i = 1$  to number of dragonflies,  $N$ 
        Calculate  $S$ ,  $A$ , and  $C$  using (1), (2), and (3)
        Compute  $F$  and  $E$  using (13) and (14)
        Update neighbouring radius
        if a dragonfly has at least one neighbouring dragonfly
            Update velocity vector using Eq. (6)
            Update position vector using Eq. (7)
        else
            Update position vector using Eq.(15)
        end if
    end for
end while

```

V. EXPERIMENTAL RESULTS AND DISCUSSIONS

A. DATA SETS

The 18 datasets that we utilised to evaluate the performance of the proposed technique are listed in Table I. They are well-known benchmark datasets from the University of California, Irvine data repository [52]. Many academics study various feature selection approaches using the same data sets. Table 1 lists the characteristics of the data sets used to evaluate the proposed improved version of HLBDA algorithm-based feature selection approach.

Table 1 Datasets used in this experiment

Data set	Features	Instances
Breastcancer	9	699
BreastEW	30	569
CongressEW	16	435
Exactly	13	1000
Exactly2	13	1000
HeartEW	13	270
IonosphereEW	34	351
KrvskpEW	36	3196
Lymphography	18	148
M-of-n	13	1000
PenglungEW	325	73
SonarEW	60	208
SpectEW	22	267
Tic-tac-toe	9	958
Vote	16	300
WaveformEW	40	5000
WineEW	13	178
Zoo	16	101

B. PARAMETER SETTING

Table 2 Parameter setting

Parameter	Value
Number of dragonflies, N	10
Maximum iterations, T	100
Number of dimensions, D	Number of features in each dataset
Personal learning rate, pl	0.5
Global learning rate, gl	0.8
Search dimension	$[-10,10] \times \text{Number of dimensions}$

C. FITNESS EVALUATION

Each dragonfly is examined iteratively using a pre-defined objective function. The objective function is used to evaluate the solutions' quality [49]. The aim function in this article is set to optimise classification accuracy while minimising the number of specified characteristics. [50, 51] The objective function is as follows:

$$Fitness = \alpha \gamma_R(D) + \beta \frac{|R|}{|C|} \quad (16)$$

where $\alpha \gamma_R(D)$ is the classification error rate of a specific classifier (in this case, the voting classifier). In addition, $|R|$ is the cardinality of the selected subset, $|C|$ is the total number of features in the dataset, α and β are two parameters related to the significance of classification quality and subset length, and $\alpha \in [0, 1]$ and $\beta = (1 - \alpha)$ are borrowed from Eq. 6.

D. EVALUATION METRICS

The datasets are randomly divided into three pieces (validation, training, and testing datasets). To guarantee that the results are strong and measurable, the data separation procedure is repeated numerous times. There are a total of 20 runs, with the validation data being used to examine the following statistical metrics in each run:

1. THE AVERAGE OF CLASSIFICATION ACCURACY:

It's a statistic that displays how accurate the classifier is given a given collection of characteristics after N runs of the algorithm, and it's calculated as follows:

$$AvgAcc = \frac{1}{N} \sum_{i=1}^N AvgAcc^i$$

Where $AvgAcc^i$ is the value of accuracy gained at run i.

2. THE AVERAGE OF SELECTED FEATURE:

It's a statistic that compares the average of selected characteristics to the total number of features when the algorithm is performed N times.

$$AvgSelection = \frac{1}{N} \sum_{i=1}^N \frac{AvgSelectionn^i}{M}$$

Where M is the total no.of features in data set and $AvgSelection^i$ is the no.of features selected at run i.

3. THE MEAN FITNESS FUNCTION:

It's a statistic that indicates the average fitness value attained at run I after the algorithm has been performed N times.

$$Mean = \frac{1}{N} \sum_{i=1}^N m^i$$

Where m^i is the mean fitness gained at run i.

4. THE BEST FITNESS FUNCTION:

It's a statistic that displays the least fitness value attained at run I after the algorithm has been performed N times.

$$Mean = \min(m)$$

Where m is the collection of best fitness over N runs.

5. THE WORST FITNESS FUNCTION:

It's a statistic that displays the highest level of fitness at run I after the algorithm has been performed N times.

$$Mean = \max(m)$$

Where m is the collection of best fitness over N runs.

6. AVERAGE COMPUTATIONAL TIME:

It is a statistic that represents the average computing time in seconds acquired when an algorithm is executed N times.

$$AvgCT = \frac{1}{N} \sum_{i=1}^N AcgCT^i$$

Where $AvgCT^i$ is the time obtained at run i.

E. RESULT AND DISCUSSION

This section contains all of the documented outcomes from the proposed FS method. When the average of selected features by enhanced HLBDA is smaller than BDA, it signifies that improved HLBDA deleted some noisy or unnecessary characteristics picked by BDA. Since the main goal of this research is to improve the performance of the Binary Dragonfy Optimizer by using a voting classifier wherein the estimators are logistic regression, decision tree, and SVM, as well as changes to the Levy Flight Mechanism (equation 15), food

(equation 13), and predator (equation 14), these methods are combined to improve the best solution (subset of features) discovered thus far., we performed further statistical research to show that the modified HLBDA strategy is superior to utilising the basic BDA alone to solve the feature selection problem.

Table 3 displays average classification accuracy, which is generally high for all data sets except HeartEW.

Table 4 displays the feature selection averages for the various FS approaches on each more robust data set. When compared to numerous well-known methods in [53-55], the number of characteristics picked using this technique is fairly minimal.

On each dataset, Table 5 displays the mean fitness for the used FS approach. In comparison to other techniques, the mean fitness is relatively low. The proposed variant also had the best mean fitness results.

The best fitness for each data set is shown in Table 6. In terms of fitness, the recommended technique delivered the best results. The greatest fitness is very modest in compared to other strategies, and it produced the best outcomes.

Table 7 displays the data sets with the worst fitness. For datasets like BreastCancer, BreastCancerEW, Exactly2, HeartEW, KrVsKp, M-of-n, SpectEW, Tic-Tac-Toe, and wine, the Worst fitness is rather high. When compared to other methodologies, the worst fitness for the vast majority of data sets is fairly low.

The computation time for each data set is shown in Table 8. When compared to other approaches, the time for all data sets, i.e. 18, is optimised. With the suggested update, the time metric is significantly lowered.

For all data sets, Table 9 provides the sensitivity, recall, F1 and other confusion matrix parameters.

A confusion matrix is a table that is often used to describe the performance of a classification model (or "classifier") on a set of test data for which the true values are known.

Confusion Matrix metrics are:

Sensitivity or Recall or hit rate or true positive rate (TPR)

It is the proportion of individuals who actually have the disease were identified as having the disease.

$$\text{TPR} = \text{tp} / (\text{tp} + \text{fn})$$

Specificity, selectivity or true negative rate (TNR)

It is the proportion of individuals who actually do not have the disease were identified as not having the disease.

$$\text{TNR} = \text{tn} / (\text{tn} + \text{fp}) = 1 - \text{FPR}$$

Precision or positive predictive value (PPV)

If the test result is positive what is the probability that the patient actually has the disease.

$$\text{PPV} = \text{tp} / (\text{tp} + \text{fp})$$

Negative predictive value (NPV)

If the test result is negative what is the probability that the patient does not have disease.

$$\text{NPV} = \text{tn} / (\text{tn} + \text{fn})$$

Miss rate or false negative rate (FNR)

It is the proportion of the individuals with a known positive condition for which the test result is negative.

$$\text{FNR} = \text{fn} / (\text{fp} + \text{tn})$$

Fall-out or false positive rate (FPR)

It is the proportion of all the people who do not have the disease who will be identified as having the disease.

$$\text{FPR} = \text{fp} / (\text{fp} + \text{tn})$$

False discovery rate (FDR)

It is the proportion of all the people identified as having the disease who do not have the disease.

$$\text{FDR} = \text{fp}/(\text{fp} + \text{tp})$$

False omission rate (FOR)

It is the proportion of the individuals with a negative test result for which the true condition is positive.

$$\text{FOR} = \text{fn}/(\text{fn} + \text{tn})$$

Accuracy

The accuracy reflects the total proportion of individuals that are correctly classified.

$$\text{ACC} = (\text{tp} + \text{tn})/(\text{tp} + \text{tn} + \text{fp} + \text{fn})$$

F1 score

It is the harmonic mean of precision and sensitivity sensitivity

$$\text{F1} = 2\text{tp}/(2\text{tp} + \text{fp} + \text{fn})$$

Table 3 Average classification accuracy

Dataset	2017_GA	2017_PSO	2017_DF	BGWOPSO	WOSAT-2	BPSO	BALO	BGWO	BDA_SA	Improved_HLBDA
BreastCancer	0.974	0.97	0.96	0.98	0.97	0.949	0.931	0.96	0.988	0.99
BreastCancerEW	0.967	0.964	0.961	0.97	0.98	0.924	0.93	0.94	0.974	0.99
CongressEW	0.967	0.964	0.967	0.98	0.98	0.912	0.915	0.92	0.975	1
Exactly	0.717	0.714	0.98	1	1	0.672	0.626	0.67	1	1
Exactly2	0.742	0.749	0.745	0.76	0.75	0.725	0.702	0.72	0.759	0.87
HeartEW	0.853	0.829	0.83	0.85	0.85	0.789	0.751	0.75	0.895	0.89
IonosphereEW	0.853	0.862	0.93	0.95	0.96	0.845	0.813	0.79	0.929	0.97
KrVsKpEW	0.953	0.953	0.953	0.98	0.98	0.85	0.761	0.86	0.976	0.99
Lymphography	0.848	0.82	0.877	0.92	0.89	0.73	0.684	0.79	0.911	0.93
M-of-n	0.892	0.927	0.992	1	1	0.814	0.733	0.8	0.997	1
PengLung	0.794	0.756	0.895	0.96	0.94	0.728	0.789	0.72	0.93	1
Sonar	0.843	0.831	0.915	0.96	0.97	0.799	0.778	0.8	0.917	0.98
SpectEW	0.865	0.856	0.853	0.88	0.88	0.808	0.831	0.79	0.866	0.91
Tic-Tac-Toe	0.765	0.769	0.788	0.81	0.79	0.712	0.694	0.71	0.818	0.87
Vote	0.96	0.96	0.958	0.97	0.97	0.905	0.882	0.92	0.964	0.99
WaveForm	0.782	0.784	0.75	0.8	0.76	0.743	0.701	0.74	0.805	0.86
Wine	0.98	0.977	0.98	1	0.99	0.916	0.916	0.9	0.999	1
Zoo	0.897	0.901	0.958	1	0.91	0.846	0.808	0.83	0.979	1

Table 4 Average Feature Selection

Dataset	2017_GA	2017_PSO	2017_DF	BGWOPSO	WOSAT-2	BPSO	BALO	BGWO	BDA_SA	Improved_HLBDA
BreastCancer	0.511	0.511	0.55	4.4	4.2	6	6	4	6	0.3
BreastCancerEW	0.46	0.44	0.395	13.6	11.6	12.8	22.6	20.6	12.45	0.29
CongressEW	0.362	0.4	0.288	4.4	6.4	3.2	9.3	9.5	4.15	0.4
Exactly	0.523	0.492	0.469	6	6	6.2	7.7	7.75	6	0.3
Exactly2	0.215	0.462	0.208	1.6	2.8	2.05	8.45	8.2	1.45	0.4
HeartEW	0.508	0.538	0.527	5.8	5.4	3.55	8.9	8.5	6.9	0.45
Ionosphere	0.418	0.424	0.338	13	12.8	12.3	25.5	20.6	10.4	0.3
KrVsKpEW	0.539	0.533	0.493	15.8	18.4	16.85	26.9	24.3	14.4	0.27
Lymphography	0.4	0.4	0.453	9.2	7.2	6.05	11.05	11.8	8.5	0.4
M-of-n	0.677	0.6	0.465	6	6	6	7.5	7.65	6.15	0.5
PengLung	0.434	0.387	0.38	130.8	127.4	146.25	253.65	205	141.3	0.12
Sonar	0.42	0.45	0.458	31.2	26.4	27	47.45	40.1	23.9	0.2
SpectEW	0.373	0.464	0.361	8.4	9.4	8	15.35	13.8	8.85	0.31
Tic-Tac-Toe	0.556	0.556	0.661	5.2	6	8.9	9	6	7.8	0.6
Vote	0.362	0.512	0.259	3.4	5.2	5	8.5	8.1	3.05	0.5
WaveForm	0.58	0.52	0.524	14.2	20.6	21.15	32.4	30.6	21.1	0.4
Wine	0.477	0.523	0.485	6	6.4	6.45	10.25	8.4	8.85	0.4
Zoo	0.362	0.4	0.356	6.8	5.6	8.15	10.55	9.35	5.65	0.4

Table 5 Mean Fitness

Dataset	2017_GA	2017_PSO	2017_DF	BGWOPSO	WOSAT-2	Improved_HLBDA
BreastCancer	0.026	0.03	0.038	0.03	0.04	0.025
BreastCancerEW	0.033	0.036	0.04	0.04	0.03	0.02
CongressEW	0.033	0.036	0.035	0.03	0.03	0.03
Exactly	0.283	0.286	0.006	0	0.0 1	0.19
Exactly2	0.258	0.251	0.249	0.24	0.25	0.23
HeartEW	0.147	0.171	0.167	0.15	0.16	0.17
Ionosphere	0.147	0.138	0.071	0.05	0.04	0.03
KrVsKpEW	0.047	0.047	0.031	0.02	0.02	0.07
Lymphography	0.152	0.18	0.125	0.08	0.11	0.13
M-of-n	0.108	0.073	0.005	0	0.0 1	0.05
PengLung	0.206	0.244	0.089	0.05	0.06	0.05
Sonar	0.157	0.169	0.066	0.05	0.03	0.08
SpectEW	0.135	0.144	0.144	0.12	0.13	0.15
Tic-Tac-Toe	0.235	0.231	0.209	0.19	0.2 1	0.16
Vote	0.04	0.04	0.041	0.03	0.04	0.03
WaveForm	0.218	0.216	0.244	0.21	0.25	0.16
Wine	0.02	0.023	0.02	0	0.0 1	0
Zoo	0.103	0.099	0.039	0	0.04	0

Table 6 Best Fitness

Dataset	2017_GA	2017_PSO	2017_DF	BGWOPSO	WOSAT-2	BPSO	BALO	BGWO	BDA_SA	Improved_HLBDA
BreastCancer	0.01	0.02	0.03	0.02	0.03	0.038	0.038	0.03	0.018	0.01
BreastCancerEW	0.021	0.026	0.025	0.03	0.02	0.044	0.036	0.05	0.03	0.01
CongressEW	0.021	0.014	0.021	0.02	0.02	0.034	0.045	0.03	0.028	0
Exactly	0.258	0.189	0.005	0	0.01	0.037	0.164	0.16	0.005	0
Exactly2	0.234	0.231	0.226	0.23	0.23	0.243	0.257	0.25	0.24	0.13
HeartEW	0.089	0.133	0.145	0.12	0.13	0.135	0.165	0.17	0.109	0.11
Ionosphere	0.12	0.111	0.042	0.04	0.03	0.113	0.141	0.18	0.074	0.02
KrVsKpEW	0.032	0.036	0.022	0.02	0.02	0.03	0.045	0.05	0.028	0.01
Lymphography	0.14	0.14	0.072	0.06	0.09	0.145	0.156	0.11	0.092	0.06
M-of-n	0.084	0.018	0.005	0	0.01	0.005	0.046	0.04	0.008	0
PengLung	0.12	0.16	0.031	0.03	0.03	0.165	0.122	0.22	0.049	0
Sonar	0.1	0.086	0.034	0.02	0.01	0.093	0.123	0.13	0.086	0.01
SpectEW	0.079	0.101	0.107	0.11	0.11	0.134	0.096	0.13	0.137	0.08
Tic-Tac-Toe	0.204	0.204	0.182	0.17	0.2	0.201	0.217	0.19	0.189	0.13
Vote	0.03	0.01	0.018	0.03	0.02	0.032	0.062	0.05	0.037	0.01
WaveForm	0.202	0.202	0.228	0.2	0.23	0.2	0.219	0.22	0.198	0.13
Wine	0.017	0.017	0.005	0	0	0.018	0.008	0.04	0.008	0
Zoo	0.063	0.069	0.001	0	0	0.045	0.033	0.08	0.024	0

Table 7 Worst Fitness

Dataset	2017_GA	2017_PSO	2017_DF	BGWOPSO	WOSAT-2	Improved_HLBDA
BreastCancer	0.03	0.039	0.04	0.03	0.04	0.05
BreastCancerEW	0.047	0.05	0.049	0.04	0.04	0.05
CongressEW	0.048	0.062	0.048	0.03	0.05	0.04
Exactly	0.312	0.342	0.023	0	0.01	0.32
Exactly2	0.27	0.27	0.286	0.25	0.27	0.3
HeartEW	0.178	0.189	0.195	0.17	0.18	0.25
Ionosphere	0.171	0.162	0.094	0.07	0.05	0.1
KrVsKpEW	0.061	0.065	0.038	0.03	0.02	0.3
Lymphography	0.18	0.22	0.167	0.1	0.14	0.19
M-of-n	0.156	0.117	0.005	0	0.01	0.2
PengLung	0.32	0.32	0.165	0.08	0.11	0.09
Sonar	0.214	0.229	0.109	0.07	0.05	0.12
SpectEW	0.18	0.191	0.197	0.13	0.15	0.26
Tic-Tac-Toe	0.26	0.26	0.238	0.2	0.23	0.29
Vote	0.05	0.08	0.069	0.05	0.04	0.03
WaveForm	0.244	0.224	0.262	0.22	0.26	0.23
Wine	0.033	0.033	0.038	0.01	0.03	0.15
Zoo	0.152	0.152	0.139	0.02	0.1	0.1

Table 8 Computation Time

Dataset	BGWOPSO	WOSAT-2	Improved_HLBDA
BreastCancer	5.69	41.74	0.01
BreastCancerEW	5.38	44.3	0.23
CongressEW	5.35	0.28	0.01
Exactly	5.61	51.79	0.18
Exactly2	5.41	54.88	0.2
HeartEW	5.25	29.79	0.2
Ionosphere	5.38	30.84	0.2
KrVsKpEW	19.61	589.56	0.2
Lymphography	5.33	26.17	0.02
M-of-n	5.6	51.54	0.03
PengLung	4.58	30.49	0.3
Sonar	5.44	27.76	0.24
SpectEW	5.71	31.38	0.02
Tic-Tac-Toe	5.3	56.89	0.18
Vote	5.72	30.89	0.01
WaveForm	33.53	1633.3	0.4
Wine	5.62	26.33	0.01
Zoo	5.22	27.02	0.01

Table 9 Confusion Matrix Metrics

Dataset	Sensitivity	NPR	FNR	FPR	FDR	FOR	F1
BreastCancer	0.95	0.97	0.029	0.021	0.041	0.028	0.95
BreastCancerEW	0.97	0.95	0.047	0.031	0.018	0.046	0.98
CongressEW	0.96	0.94	0.06	0.04	0.025	0.058	0.97
Exactly	0.97	0.83	0.063	0.68	0.24	0.16	0.85
Exactly2	0.95	0.76	0.29	0.03	0.5	0.23	0.16
HeartEW	0.75	0.81	0.2	0.13	0.18	0.187	0.78
Ionosphere	0.89	0.94	0.059	0	0	0.056	0.94
KrVsKpEW	0.98	0.99	0.011	0.017	0.019	0.012	0.98
Lymphography	1	1	0	0	0	0	1
M-of-n	1	1	0	0	0	0	1
PengLung	1	1	0	0	0	0	1
Sonar	0.85	0.82	0.172	0.206	0.171	0.178	0.84
SpectEW	0.47	0.86	0.142	0.047	0.27	0.13	0.57
Tic-Tac-Toe	0.81	0.9	0.101	0	0	0.09	0.9
Vote	0.94	0.96	0.036	0.018	0.029	0.035	0.96
WaveForm	0.83	0.91	0.087	0.075	0.151	0.086	0.84
Wine	1	1	0	0	0	0	1
Zoo	1	1	0	0	0	0	1

VI. CONCLUSION

In this paper, an improved HLBDA is suggested and applied to overcome the problem of feature selection. 18 common UCI benchmark datasets were utilised to confirm the usefulness and efficiency of the proposed technique. The suggested technique was evaluated using a set of evaluation measures. The suggested improved HLBDA was compared to the GA [53], PSO [53], DF [53], BGWOPSO [54], WOSAT-2 [54], BPSO [55], BALO [55], BGWO [55], and BDA SA [55] feature selection algorithms.

The findings demonstrated that the proposed strategy outperformed a wide range of methods in terms of accuracy and number of features picked on the majority of datasets. A computational time comparison was also performed between the proposed enhanced HLBDA approach, BGWOPSO, and WOSAT-2, with the results demonstrating that the proposed method has a quicker execution time. A comparison of statistical measures Mean, Best, and Worst Fitness was also conducted, with the results revealing that the proposed technique beat existing state-of-the-art methodologies significantly. The superior results of the recommended enhanced HLBDA approach indicate that it can control the trade-off between exploratory and exploitative inclinations throughout optimisation rounds.

In future study, we advocate applying the proposed enhanced version of the HLBDA technique to handle different real-world concerns such as engineering optimisation challenges, scheduling constraints, and/or molecular potential energy function. Furthermore, the proposed technique for Feature Selection may be compared to other well-known classifiers, such as the Artificial Neural Network (ANN), to see if performance remains consistent or varies.

VII .REFERENCES

1. Kowal M, Skobel M, Nowicki N (2018) The feature selection problem in computer-assisted cytology. *Int Jou Appl Math Comput Sci* 28(4):759–770. <https://doi.org/10.2478/amcs-2018-0058>
2. Borowik G (2018) Optimization on the complementation procedure towards efficient implementation of the index generation function. *Int J Appl Math Comput Sci* 28(4):803–815. <https://doi.org/10.2478/amcs-2018-0061>
3. Ghosh KK et al (2020) A two-stage approach towards protein secondary structure classification. *Med Biol Eng Comput* 58(8):1723–1737. <https://doi.org/10.1007/s11517-020-02194-w>
4. Han J, Pei J, Kamber M (2011) *Data mining: concepts and techniques*. Elsevier, Amsterdam
5. Kumar N, Makkar A (2020) *Machine learning in cognitive IoT*. CRC Press, Boca Raton
6. Miglani A, Kumar N (2019) Deep learning models for traffic flow prediction in autonomous vehicles: a review, solutions, and challenges. *Veh Commun* 20:100184. <https://doi.org/10.1016/j.Vehcom.2019.100184>
7. Garg S et al (2019) Hybrid deep-learning-based anomaly detection scheme for suspicious flow detection in SDN: a social multimedia perspective. *IEEE Trans Multimed* 21(3):566–578. <https://doi.org/10.1109/tmm.2019.2893549>
8. Garg S et al (2019) A hybrid deep learning-based model for anomaly detection in cloud datacenter networks. *IEEE Trans Netw Serv Manag* 16(3):924–935. <https://doi.org/10.1109/tnsm.2019.2927886>
9. Liu H, Motoda H (2012) *Feature selection for knowledge discovery and data mining*, vol 454. Spr
10. Guha R et al. (2020) “Mutually informed correlation coefficient (MICC)—a new filter based feature selection method. In: 2020 IEEE Calcutta Conference (CALCON), pp. 54–58. <https://doi.org/10.1109/CALCON49167.2020.9106516>
11. Blachnik M (2019) Ensembles of instance selection methods: a comparative study. *Int J Appl Math Comput Sci* 29(1):151–168. <https://doi.org/10.2478/amcs-2019-0012>

12. Guha R et al (2020) Embedded chaotic whale survival algorithm for filter-wrapper feature selection. *Soft Comput* 24(17):12821–12843. <https://doi.org/10.1007/s00500-020-05183-1>
13. Guha R et al (2020) Introducing clustering based population in binary gravitational search algorithm for feature selection. *Appl Soft Comput* 93:106341. <https://doi.org/10.1016/j.asoc.2020.106341>
14. Balochian S, Baloochian H (2019) Social mimic optimization algorithm and engineering applications. *Expert Syst Appl* 134:178–191. <https://doi.org/10.1016/j.eswa.2019.05.035>
15. Ghosh M et al (2018) Genetic algorithm based cancerous gene identification from microarray data using ensemble of filter methods. *Med Biol Eng Comput* 57(1):159–176. <https://doi.org/10.1007/s11517-018-1874-4>
16. Al-Tashi Q et al (2019) Binary optimization using hybrid grey wolf optimization for feature selection. *IEEE Access* 7:39496–39508
17. Mafarja MM, Mirjalili S (2019) Hybrid binary ant lion optimizer with rough set and approximate entropy reducts for feature selection. *Soft Comput* 23(15):6249–6265
18. Il-Seok O, Lee J-S, Moon B-R (2004) Hybrid genetic algorithms for feature selection. *IEEE Trans Pattern Anal Mach Intell* 26(11):1424–1437. <https://doi.org/10.1109/tpami.2004.105>
19. Chen H et al (2013) A heuristic feature selection approach for text categorization by using chaos optimization and genetic algorithm. *Math Probl Eng*. <https://doi.org/10.1155/2013/524017>
20. Kennedy J, Eberhart R (1995) Particle swarm optimization. In: *Proceedings of ICNN'95-international conference on neural networks*. vol 4. IEEE, pp 1942–1948
21. Lee S et al (2008) Modified binary particle swarm optimization. *Progress Nat Sci* 18(9):1161–1166. <https://doi.org/10.1016/j.pnsc.2008.03.018>
22. Wang X et al (2007) Feature selection based on rough sets and particle swarm optimization. *Pattern Recognit Lett* 28(4):459–471
23. Chakraborty B (2008) Feature subset selection by particle swarm optimization with fuzzy fitness function. In: *2008 3rd international conference on intelligent system and knowledge engineering*. vol 1, pp 1038–1042. <https://doi.org/10.1109/ISKE.2008.4731082>

24. Dorigo M, Maniezzo V, Colorni A (1996) Ant system optimization by a colony of cooperating agents. *IEEE Trans Syst Man Cybern Part B (Cybern)* 26(1):29–41
25. Chen Y, Miao D, Wang R (2010) A rough set approach to feature selection based on ant colony optimization. *Pattern Recognit Lett* 31(3):226–233. <https://doi.org/10.1016/j.patrec.2009.10.013>
26. Ke L, Feng Z, Ren Z (2008) An efficient ant colony optimization approach to attribute reduction in rough set theory. *Pattern Recognit Lett* 29(9):1351–1357
27. Karaboga D (2005) An idea based on honey bee swarm for numerical optimization. Tech. rep. Technical report-tr06, Erciyes University, Engineering Faculty, Computer Engineering Department
28. Shokouhifar M, Sabet S (2010) A hybrid approach for effective feature selection using neural networks and artificial bee colony optimization. In: 3rd international conference on machine vision (ICMV 2010), pp 502–506
29. Ghosh KK et al (2020) Binary social mimic optimization algorithm with X-shaped transfer function for feature selection. *IEEE Access* 8:97890–97906. <https://doi.org/10.1109/ACCESS.2020.2996611>
30. Mafarja M, Mirjalili S (2018) Whale optimization approaches for wrapper feature selection. *Appl Soft Comput* 62:441–453
31. Rashedi E, Nezamabadi-Pour H, Saryazdi S (2010) BGSA: binary gravitational search algorithm. *Nat Comput*
32. Chatterjee B et al (2020) Late acceptance hill climbing based social ski driver algorithm for feature selection. *IEEE Access* 8:75393–75408. <https://doi.org/10.1109/ACCESS.2020.2988157>
33. Ghosh KK et al (2020) Improved binary sailfish optimizer based on adaptive b-Hill climbing for feature selection. *IEEE Access* 8:83548–83560. <https://doi.org/10.1109/access.2020.2991543>
34. Ahmed S et al (2020) Hybrid of harmony search algorithm and ring theory- based evolutionary algorithm for feature selection. *IEEE Access* 8:102629–102645. <https://doi.org/10.1109/ACCESS.2020.2999093>
35. S. Mirjalili, "Dragonfly algorithm: a new metaheuristic optimization technique for solving singleobjective, discrete, and multi-objective problems," *Neural Computing and Applications*, vol. 27, pp. 1053- 1073, 2016.

36. C. W. Reynolds, "Flocks, herds and schools: A distributed behavioral model," ACM SIGGRAPH computer graphics, vol. 21, pp. 25-34, 1987.
37. Yang X-S (2010) Nature-inspired metaheuristic algorithms, 2nd edn. Luniver Press
38. Kadrovach BA, Lamont GB (2002) A particle swarm model for swarm-based networked sensor systems. In: Proceedings of the 2002 ACM symposium on applied computing, pp 918–924
39. Cui Z (2009) Alignment particle swarm optimization. In: Cognitive informatics, 2009. ICCI'09. 8th IEEE international conference on, pp 497–501
40. Cui Z, Shi Z (2009) Boid particle swarm optimisation. Int J Innov Comput Appl 2:77–85
41. Mirjalili S, Lewis A (2013) S-shaped versus V-shaped transfer functions for binary particle swarm optimization. Swarm Evolut Comput 9:1–14
42. Mirjalili S, Wang G-G, Coelho LDS (2014) Binary optimization using hybrid particle swarm optimization and gravitational search algorithm. Neural Comput Appl 25:1423–1435
43. Saremi S, Mirjalili S, Lewis A (2014) How important is a transfer function in discrete heuristic algorithms. Neural Comput Appl:1–16
44. X. Yang and S. Deb, "Cuckoo search via Lévy flights," in *Proceedings of the 2009 World Congress on Nature & Biologically Inspired Computing (NaBIC)*, Coimbatore, India, December 2009.
45. R. Rajakumar, P. Dhavachelvan, and T. Vengattaraman, "A survey on nature inspired meta-heuristic algorithms with its domain specifications," in *Proceedings of the 2016 International Conference on Communication and Electronics Systems (ICCES)*, Coimbatore, India, October 2016.
46. H. Chiroma, T. Herawan, I. Fister et al., "Bio-inspired computation: recent development on the modifications of the cuckoo search algorithm," *Applied Soft Computing*, vol. 61, pp. 149–173, 2017.

47. T. Dokeroglu, E. Sevinc, T. Kucukyilmaz, and A. Cosar, "A survey on new generation metaheuristic algorithms," *Computers & Industrial Engineering*, vol. 137, p. 106040, 2019.
48. H. M. Mohammed, S. U. Umar, and T. A. Rashid, "A Systematic and Meta-Analysis Survey of Whale Optimization Algorithm," *Computational Intelligence and Neuroscience*, vol. 2019, Article ID 8718571, 25 pages, 2019.
49. Sayed GI, Tharwat A, Hassanien AE (2019) Chaotic dragonfly algorithm: an improved metaheuristic algorithm for feature selection. *Appl Intell* 49:188–205. <https://doi.org/10.1007/s10489-018-1261-8>
50. Emary E, Zawbaa HM, Hassanien AE (2016) Binary ant lion approaches for feature selection. *Neurocomputing* 213:54–65. <https://doi.org/10.1016/j.neucom.2016.03.101>
51. Mafarja MM, Eleyan D, Jaber I, et al (2017) Binary Dragonfly Algorithm for Feature Selection. In: 2017 International Conference on New Trends in Computing Sciences (ICTCS). pp 12–17
52. C. L. Blake and C. J. Merz. (1998, 1 June). UCI Repository of machine learning databases. Available: <http://www.ics.uci.edu/~mlearn/>
53. Mafarja, M. M., Eleyan, D., Jaber, I., Hammouri, A., & Mirjalili, S. (2017). *Binary Dragonfly Algorithm for Feature Selection. 2017 International Conference on New Trends in Computing Sciences (ICTCS)*. doi:10.1109/ictcs.2017.43
54. Al-Tashi, Q., Abdulkadir, S. J., Rais, H. M., Mirjalili, S., & Alhussian, H. (2019). *Binary Optimization Using Hybrid Grey Wolf Optimization for Feature Selection. IEEE Access, 1–1*. doi:10.1109/access.2019.2906757
55. Chantar, H., Tubishat, M., Essgaer, M., & Mirjalili, S. (2021). *Hybrid Binary Dragonfly Algorithm with Simulated Annealing for Feature Selection. SN Computer Science, 2(4)*. doi:10.1007/s42979-021-00687-5
56. Wang J, Yang Y, Wang T, et al (2020) Big Data Service Architecture: A Survey. *J Internet Technol* 21:393–405
57. Xue Y, Tang T, Pang W, Liu AX (2020) Self-adaptive parameter and strategy based particle swarm optimization for large-scale feature selection problems with multiple classifiers. *Appl Soft Comput* 88:106031. <https://doi.org/10.1016/j.asoc.2019.106031>

58. Ji B, Lu X, Sun G, et al (2020) Bio-Inspired Feature Selection: An Improved Binary Particle Swarm Optimization Approach. IEEE Access 8:85989–86002. <https://doi.org/10.1109/ACCESS.2020.2992752>
59. Qiu C (2019) A novel multi-swarm particle swarm optimization for feature selection. Genet Program Evolvable Mach. http Journal Pre-proof [s://doi.org/10.1007/s10710-019-09358-0](https://doi.org/10.1007/s10710-019-09358-0)
60. Mirjalili S, Mirjalili SM, Lewis A (2014) Grey Wolf Optimizer. Adv Eng Softw 69:46– 61. <https://doi.org/10.1016/j.advengsoft.2013.12.007>
61. Emary E, Zawbaa HM, Hassanien AE (2016) Binary grey wolf optimization approaches for feature selection. Neurocomputing 172:371–381. <https://doi.org/10.1016/j.neucom.2015.06.083>
62. Zawbaa HM, Emary E, Grosan C, Snasel V (2018) Large-dimensionality small-instance set feature selection: A hybrid bio-inspired heuristic approach. Swarm Evol Comput 42:29–42. <https://doi.org/10.1016/j.swevo.2018.02.021>
63. Chantar H, Mafarja M, Alsawalqah H, et al (2019) Feature selection using binary grey wolf optimizer with elite-based crossover for Arabic text classification. Neural Comput Appl. <https://doi.org/10.1007/s00521-019-04368-6>
64. Too J, Abdullah AR (2020) Opposition based competitive grey wolf optimizer for EMG feature selection. Evol Intell. <https://doi.org/10.1007/s12065-020-00441-5>
65. Abdel-Basset M, El-Shahat D, El-henawy I, et al (2020) A new fusion of grey wolf optimizer algorithm with a two-phase mutation for feature selection. Expert Syst Appl 139:112824. <https://doi.org/10.1016/j.eswa.2019.112824>
66. Mafarja M, Mirjalili S (2018) Whale optimization approaches for wrapper feature selection. Appl Soft Comput 62:441–453. <https://doi.org/10.1016/j.asoc.2017.11.006>
67. Mafarja MM, Mirjalili S (2017) Hybrid Whale Optimization Algorithm with simulated annealing for feature selection. Neurocomputing 260:302–312. <https://doi.org/10.1016/j.neucom.2017.04.053>
68. Tubishat M, Abushariah MAM, Idris N, Aljarah I (2019) Improved whale optimization algorithm for feature selection in Arabic sentiment analysis. Appl Intell 49:1688–1707. <https://doi.org/10.1007/s10489-018-1334-8>
69. Mirjalili S (2016) Dragonfly algorithm: a new meta-heuristic optimization technique for solving single-objective, discrete, and multi-objective problems. Neural Comput Appl 27:1053–1073. <https://doi.org/10.1007/s00521-015-1920-1>

70. Mafarja MM, Eleyan D, Jaber I, et al (2017) Binary Dragonfly Algorithm for Feature Selection. In: 2017 International Conference on New Trends in Computing Sciences (ICTCS). pp 12–17
71. Sayed GI, Tharwat A, Hassanien AE (2019) Chaotic dragonfly algorithm: an improved metaheuristic algorithm for feature selection. Appl Intell 49:188–205. <https://doi.org/10.1007/s10489-018-1261-8>